



早稲田大学
WASEDA University

WASEDA University

Waseda Research Institute for Science and Engineering

Fundamentals of Programming

孫 鶴鳴

SUN Heming

May 12th



Schedule

- ◆ May 26
 - ❖ Exercise
- ◆ May 29
 - ❖ Q & A
- ◆ June 2
 - ❖ Final exam

Outline

◆ Function

◆ Pointer

Function

- ◆ A function is a set of statements that take inputs, do some specific computation and produces output.

A function definition has this form:

return-type function-name(parameter declarations, if any)

{declarations

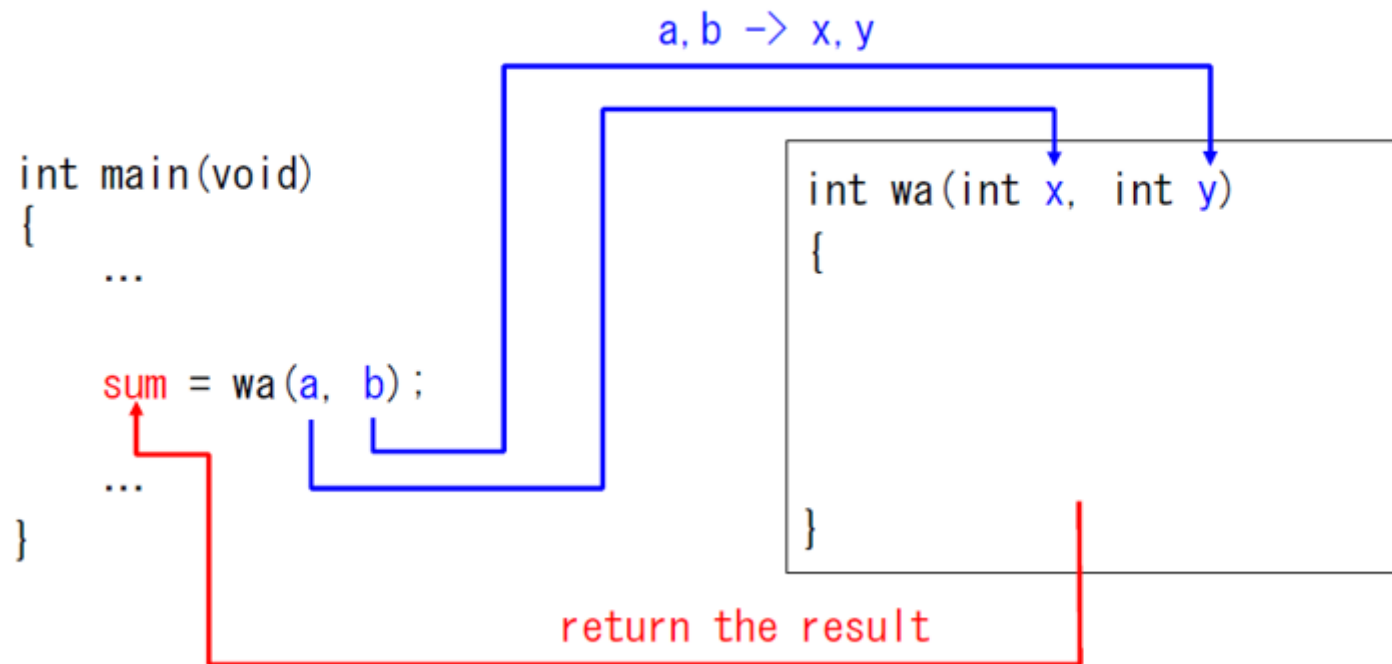
statements }

Question:

How many returns you can have in a function?

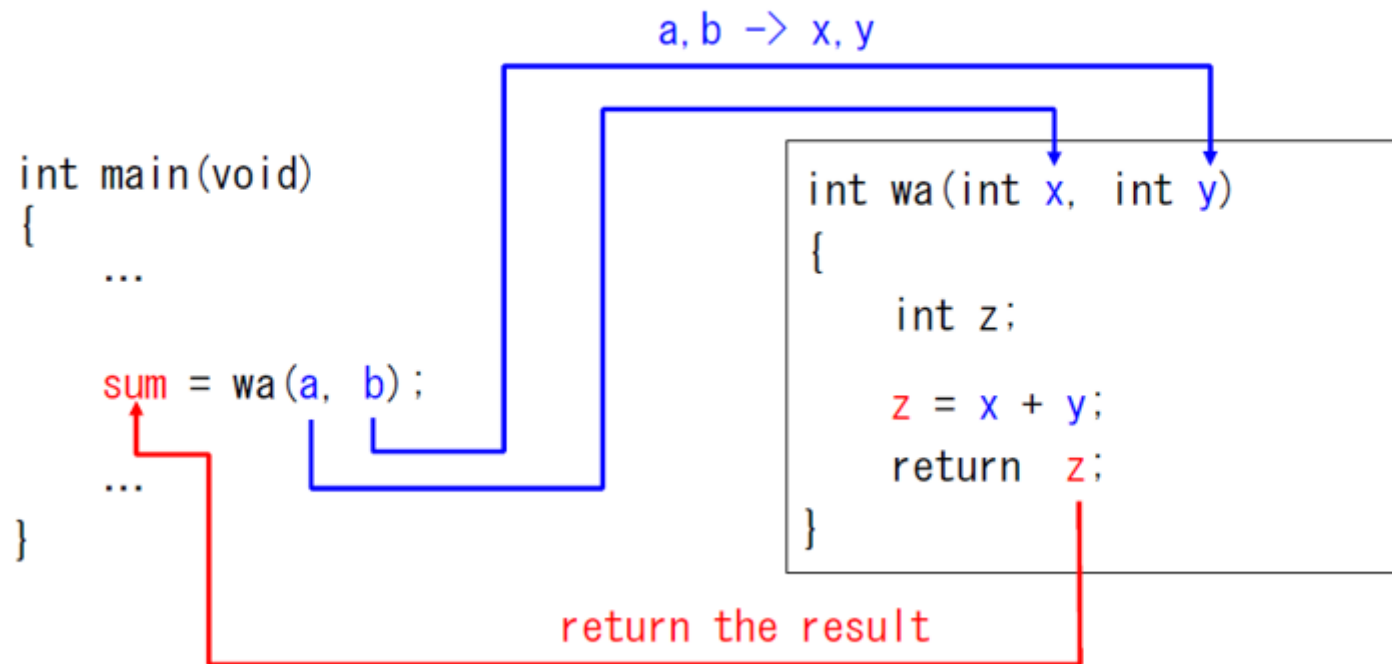
Calling Function

- In C programming, the execution of program is started from main()



Calling Function

- In C programming, the execution of program is started from main()



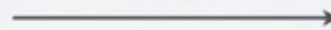
Exercise 1

```
#include <stdio.h>
```

```
int main(){  
    a = sum(12, 23);
```

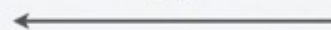
```
    printf("%d\n", a);  
}
```

12, 23



```
int sum(int x, int y){  
    return x + y;  
}
```

35



Make a “sum” function

Exercise 1 (cont.)

```
#include <stdio.h>

int sum(int x, int y){
    return x+y;
}

int main(){

    int a=30;
    int b=2;
    int c=sum(a,b);

    printf("end of this function¥n");

    return 0;
    printf("%d¥n",c);
}
```

```
#include <stdio.h>
```

```
int sum();
```

Function declaration

```
int main(){
```

```
    int a=30;
```

```
    int b=2;
```

```
    int c=sum(a,b);
```

```
    printf("end of this function¥n");
```

```
    return 0;
```

```
    printf("%d¥n",c);
```

```
}
```

Function definition

```
int sum(int x, int y){
```

```
    return x+y;
```

```
}
```


Outline

- ◆ Function
- ◆ Pointer

Pointers

- ◆ Pointer is a variable that stores/points to the address of another variable.
- ◆ The pointer variable might belong to any data type such as int, float, char, double and so on.

To get the address of variables

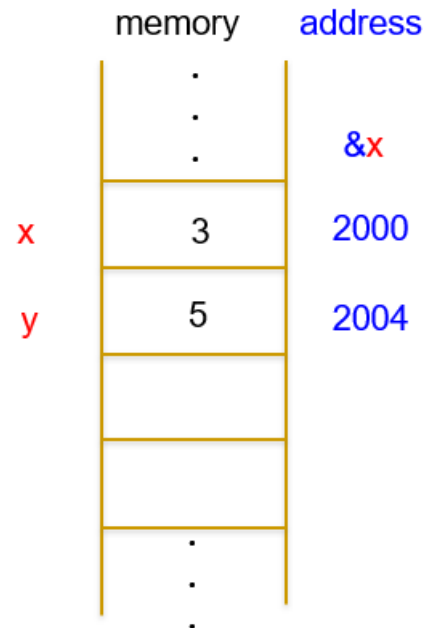
For examples

```
int x, y;  
x=3;  
y=5;
```

by writing `&x`

we can get the address of variable x

`&` is called address operator



Pointers (cont.)

Pointers

A **pointer** is a variable that contains the **address** of a variable

To declare a pointer

For examples

int ***x_p**: * is called indirection operator

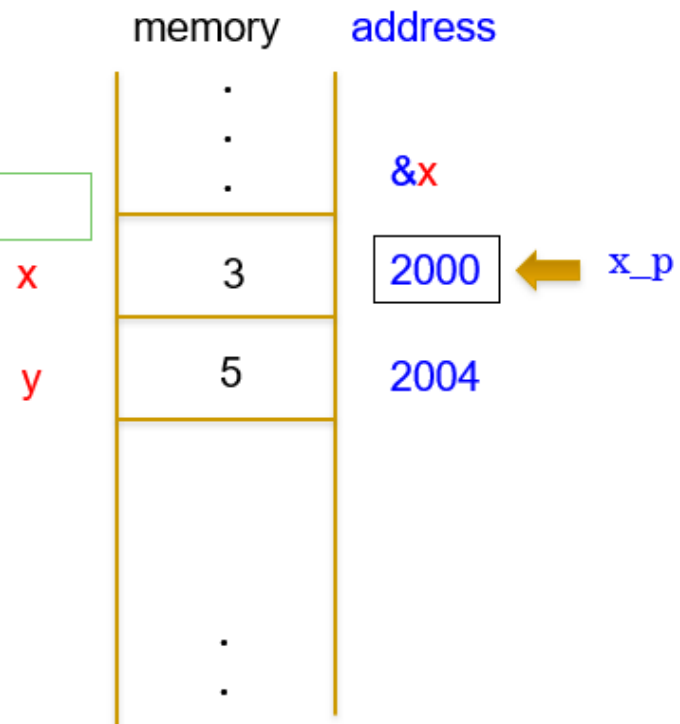
x_p will be a pointer to show

the address of a variable (int type)

In this case,

x_p = **&x**;

pointer **x_p** will store the address of x
That is to say: [**x_p** **points to** x]



Type of Pointers

■ A pointer can be declared as int or double type

■ But the pointer is not independent, we call 「the pointer to int type」 or 「the pointer to double type」 .

Ex int *p_a; double *p_b;

- An int (four bytes) will be 4-byte aligned
- A double (eight bytes) will be 8-byte aligned

(Reference)

- 1Kbyte == 1024byte
- 1Mbyte == 1024Kbyte $1024 == 2^{10}$
- 1Gbyte == 1024Mbyte

some mistakes:

```
float a;
```

```
int * pointer_1;
```

```
pointer_1=&a;
```



Get The Address of a Variable

```
#include <stdio.h>
```

pointer1.c

```
int main(void)
```

```
{
```

```
    int x = 3;
```

```
    int y = 5;
```

```
    printf("x..%d, y..%d\n", x, y);
```

```
    printf("x..%p, y..%p\n", &x, &y);
```

```
    return 0;
```

```
}
```

show the integer by decimal number

integer

pointer

to show the pointer

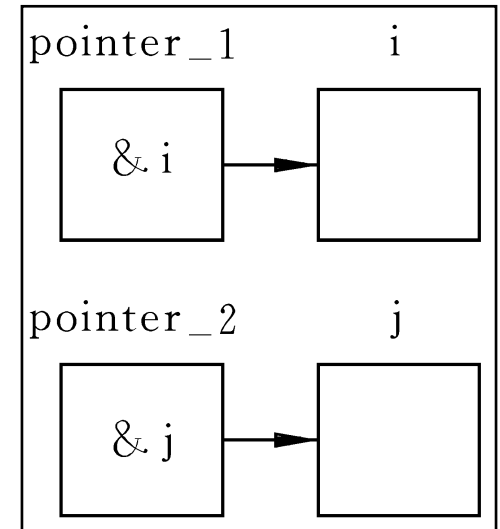
Pointers

```
#include <stdio.h>
int main ()
{ int i, j;
  int *pointer_1, *pointer_2;
  i=3;
  j=5;

  / pointer_1 now pointes to i /
  / pointer_2 now pointes to j /

  printf("%d, %d\n", i, j);
  printf("%d, %d\n", *pointer_1, *pointer_2);
  return 0;
}
```

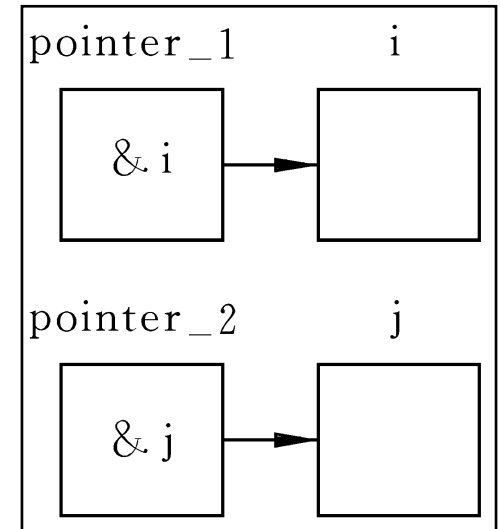
Pointer2.c



Pointers

```
#include <stdio.h>
int main ()
{ int i, j;
  int *pointer_1, *pointer_2;
  i=3;
  j=5;
  pointer_1 = &i; / pointer_1 now pointes to i /
  pointer_2 = &j; / pointer_2 now pointes to j /
  printf("%d, %d\n", i, j);
  printf("%d, %d\n", *pointer_1, *pointer_2);
  return 0;
}
```

Pointer2.c



Exercise 2

```
#include <stdio.h>
```

```
pointer3.c
```

```
int main(void)
```

```
{
```

```
    int a = 123, b;
```

```
    int *p;
```

```
    p = &a;
```

```
    b = *p;
```

```
    printf("a = %d, b = %d, *p = %d\n", a, b, *p);
```

```
    a = 200;
```

```
    b = 300;
```

```
    printf("a = %d, b = %d, *p = %d\n", a, b, *p);
```

Let p points to b, and then print *p

```
    return 0;
```

```
}
```


Exercise 2 (cont.)

```
#include <stdio.h>
```

pointer3.c

```
int main(void)
```

```
{
```

```
    int a = 123, b;
```

```
    int *p;
```

p points to a, *p=a

```
    p = &a;
```

```
    b = *p;
```

b = a

```
    printf("a = %d, b = %d, *p = %d\n", a, b, *p);
```

```
    a = 200;
```

```
    b = 300;
```

```
    printf("a = %d, b = %d, *p = %d\n", a, b, *p);
```

p does not change
(points to a)

```
    p = &b;
```

```
    printf("a = %d, b = %d, *p = %d\n", a, b, *p);
```

p points to b

```
    return 0;
```

```
}
```

Pointer Arithmetic

Pointers can be calculated as follows

- add, subtraction

$p_a + 1$

$p_b + 2$

- increment, decrement

$p_a ++$

$p_b --$

- Pointer comparisons

$p_a == p_b$

$p_a < p_b$

$p_a > p_b$

Exercise 3

pointer4.c

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int a;
```

```
    int *p_a;
```

```
    double b;
```

```
    double *p_b;
```

```
    p_a = &a;
```

```
    printf("p_a ... %p\n", p_a);
```

```
    p_a++;
```

```
    printf("p_a ... %p\n", p_a);
```



```
    p_b = &b;
```

```
    printf("p_b ... %p\n", p_b);
```

```
    p_b++;
```

```
    printf("p_b ... %p\n", p_b);
```

```
    return 0;
```



```
}
```

Exercise 3 (cont.)

`p_a++;` ← add 1 to `p_a(int)`

`p_b++;` ← add 1 to `p_b(double)`

「pointer4.c」' result....

```
— $ ./pointer4 ↵  
p_a ... 0xbffff6a4  
p_a ... 0xbffff6a8   ↘ +4  
p_b ... 0xbffff698  
p_b ... 0xbffff6a0   ↘ +8
```

the address of pointers,

「 increment by 4 for int poniter,
 increment by 8 for double poniter」